

1. Discuss the various states in which a process can exist. Draw the process state diagram and explain the transitions among the various states.

Process State

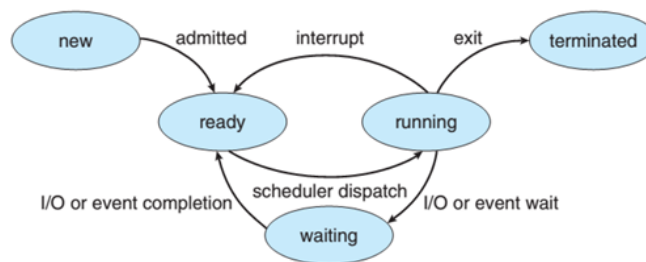
As a process executes, it changes state. The process state defines the current activity of that process.

A process may be in one of the following states:

1. **New** – The process is being created.
2. **Ready** – The process is waiting to be assigned to a processor.
3. **Running** – Instructions are being executed.
4. **Waiting (Blocked)** – The process is waiting for some event to occur such as I/O completion or reception of a signal.
5. **Terminated** – The process has finished execution.

Note: Only one process can be running on any processor at any instant of time.

Process State Transition Diagram



Explanation of State Transitions

1. New → Ready (Admitted)

When a process is created, it enters the New state. After the operating system admits it into the ready queue, it moves to the Ready state.

2. Ready → Running (Scheduler Dispatch)

The process in the ready queue waits for CPU allocation. When the CPU scheduler selects it, the process moves from Ready to Running state.

3. Running → Waiting (I/O or Event Wait)

While executing, a process may request an I/O operation or wait for an event. In such cases, it cannot continue execution and moves to the Waiting state.

4. Waiting → Ready (I/O Completion)

After the required I/O operation or event completes, the process returns to the Ready state and waits again for CPU allocation.

5. Running → Ready (Interrupt)

A running process may be interrupted due to time-slice expiration or a higher-priority process. It is then moved back to the Ready state.

6. Running → Terminated (Exit)

When a process completes execution, it enters the Terminated state and is removed from the system.

2. Explain the operations on processes: (a) Process Creation (b) Process Termination

Operations on Processes

The two major operations performed on a process are:

1. Process Creation
2. Process Termination

(a) Process Creation

During its execution, a process may create several new processes.

- The creating process is called the Parent Process.
- The newly created processes are called Child Processes.
- Each child process may create additional processes, forming a tree of processes.
- The operating system identifies each process using a unique Process Identifier (PID).
- PID provides a unique integer number for every process in the system and is used to access process attributes within the kernel.

Resource Sharing During Process Creation

When a child process is created, it requires resources such as:

- CPU time
- Memory
- Files
- I/O devices

A child process may:

- Obtain resources directly from the operating system, or
- Use a subset of the parent's resources.

Restricting resource allocation prevents any process from overloading the system by creating too many child processes.

Execution Possibilities

When a new process is created, two possibilities exist:

1. **Parent and child execute concurrently.**
2. **Parent waits until some or all child processes terminate.**

Address Space Possibilities

1. **Child process is a duplicate of the parent process.**
2. **Child process has a new program loaded into it.**

UNIX/Linux System Calls Used

fork()

- Creates a new process.
- Child process receives a copy of the parent's address space.
- Both parent and child continue execution after the fork() call.
- fork() returns 0 to the child process.
- Parent receives the PID of the child process.

exec()

- Replaces the process memory with a new program.
- Loads the binary file into memory and starts execution.

wait()

- Causes the parent process to wait until the child process completes execution.

(b) Process Termination

A process terminates when it completes execution and requests the operating system to delete it using the exit() system call.

During termination:

- The process may return a status value to its parent.
- All resources allocated to the process are released.
- Physical memory, virtual memory, open files and I/O buffers are deallocated by the operating system.

Reasons for Terminating a Child Process

A parent process may terminate a child process because:

1. **The child exceeded its allocated resources.**
2. **The assigned task is no longer required.**
3. **The parent is exiting and the operating system does not allow the child to continue.**

Cascading Termination

If a parent process terminates, all its child processes are also terminated.

This phenomenon is called Cascading Termination.

Zombie Process

A process that has completed execution but whose parent has not yet called wait() is called a Zombie Process.

- When a process terminates, its resources are deallocated by the operating system. Its entry in the process table must remain there until the parent calls wait(), because the process table contains the process's exit status.
- Once the parent calls wait(), the process identifier of the zombie process and its entry in the process table are released.

Orphan Process

If a parent terminates before its child process, the child becomes an Orphan Process.

- In UNIX/Linux systems, the init process (PID = 1) becomes the new parent.
- The init process periodically invokes wait() and removes orphan processes.

3. Explain the Different Multithreading Models

Multithreading Models

To execute user threads on a CPU, user-level threads must be mapped to kernel threads. The relationship between user threads and kernel threads is defined by Multithreading Models.

The three common multithreading models are:

1. Many-to-One Model
2. One-to-One Model
3. Many-to-Many Model

1. Many-to-One Model

In this model, many user-level threads are mapped to a single kernel thread.

Diagram

User Threads

T1

T2

T3 ----- Single Kernel Thread -CPU

T4

T5

Advantages

- Thread management is done in user space.
- Efficient and fast thread creation.

Disadvantages

- If one thread performs a blocking system call, the entire process blocks.
- Multiple threads cannot run in parallel on multiprocessor systems.

2. One-to-One Model

In this model, each user thread is mapped to a separate kernel thread.

Diagram

User Thread T1 ---> Kernel Thread K1

User Thread T2 ---> Kernel Thread K2

User Thread T3 ---> Kernel Thread K3

User Thread T4 ---> Kernel Thread K4

Advantages

- Allows multiple threads to run in parallel on multiprocessor systems.
- A blocking system call by one thread does not block other threads.

Disadvantages

- Creating many kernel threads can affect system performance.
- More overhead due to kernel thread management.

Examples

- Windows
- Linux
- Solaris

3. Many-to-Many Model

In this model, many user threads are multiplexed over a smaller or equal number of kernel threads.

Diagram

User Threads

T1

T2 ----- Kernel Threads

T3 ----- K1 K2 K3 ---- CPU

T4

T5

Advantages

- Supports a large number of user threads.
- Allows true parallelism.
- Blocking of one thread does not affect others.
- Better performance and flexibility.

Disadvantages

- More complex to implement.

Comparison of Multithreading Models

Feature	Many-to-One	One-to-One	Many-to-Many
Kernel Threads	One	One per user thread	Multiple
Parallel Execution	No	Yes	Yes
Blocking Problem	Entire process blocks	No	No
Performance	High	Moderate	High
Complexity	Simple	Moderate	Complex

4. Explain Thread State Cycle Diagram

Thread States

A thread passes through various states during its lifetime. The thread state cycle describes the movement of a thread from creation to termination.

Thread State Diagram

Waiting(Blocked) -----> Ready

| New | → | Ready | → | Running | → Ready

Terminated(Exit)

Explanation of States

1. New State

- A thread is created but has not yet started execution.
- Required resources are allocated.

2. Ready State

- The thread is ready to run and is waiting for CPU allocation.
- It remains in the ready queue until selected by the scheduler.

3. Running State

- The thread is currently executing on the CPU.
- Only one thread can run on a processor at a time.

4. Waiting (Blocked) State

- A running thread may need to wait for an event such as:
 - I/O completion
 - Resource availability
 - Signal reception
- During this period it cannot execute.

5. Terminated State

- The thread completes execution and exits.
- All resources allocated to the thread are released.

State Transitions

- **New → Ready**: Thread is admitted for execution.
- **Ready → Running**: CPU scheduler allocates CPU.
- **Running → Waiting**: Thread waits for an event or I/O operation.
- **Waiting → Ready**: Event completes and thread becomes ready.
- **Running → Ready**: CPU is preempted or time slice expires.
- **Running → Terminated**: Thread finishes execution.

5. What is a Thread? Explain the Benefits of Multithreaded Programming.

Thread

A thread is the basic unit of CPU utilization.

A thread consists of:

- Thread ID
- Program Counter
- Register Set
- Stack

A thread shares with other threads belonging to the same process:

- Code Section
- Data Section
- Open Files and Resources

In a traditional process, there is only one thread of execution. In a multithreaded process, multiple threads execute concurrently within the same process.

Multithreading

Multithreading is the ability of a process to execute multiple threads concurrently.

In a multicore or multiprocessor system, multiple threads can execute in parallel, improving system performance.

Benefits of Multithreaded Programming

1. Responsiveness

Multithreading allows a program to continue running even if part of it is blocked or performing a lengthy operation.

Example: In a word processor, a user can continue typing while the spell checker runs in the background.

2. Resource Sharing

Threads belonging to the same process share memory and resources.

- Shared code section
- Shared data section
- Shared files

This makes communication between threads easier and faster.

3. Economy

Creating and managing threads is less expensive than creating and managing processes.

- Thread creation requires less overhead.
- Context switching between threads is faster than between processes.

4. Scalability

Multithreading takes advantage of multiprocessor and multicore architectures.

- Multiple threads can run simultaneously on different processors.
- Improves CPU utilization and throughput.

Single Thread vs Multithread

Single Thread	Multithread
Performs one task at a time	Performs multiple tasks concurrently
Lower CPU utilization	Better CPU utilization
Less responsive	More responsive
No parallel execution	Supports parallel execution

6. What is Pthread? Explain with an Example Program.

Pthread

Pthread (POSIX Thread) is a standard API used for creating and managing threads in UNIX and Linux operating systems.

POSIX stands for Portable Operating System Interface.

Pthreads provide a set of functions that allow programmers to:

- Create threads
- Terminate threads
- Synchronize threads
- Control thread execution

Pthreads are commonly used for multithreaded programming in UNIX and Linux systems.

Common Pthread Functions

1. pthread_create()

Used to create a new thread.

Syntax:

```
pthread_create(thread,attr,start_routine,arg);
```

2. pthread_exit()

Used to terminate a thread.

Syntax:

`pthread_exit(NULL);`

3. pthread_join()

Used to make one thread wait for another thread to complete.

Syntax:

`pthread_join(thread, NULL);`

Example Program Using Pthread

```
#include <pthread.h>

#include <stdio.h>

void *print_message(void *arg)
{
    printf("Hello from Thread\n");
    pthread_exit(NULL);
}

int main()
{
    pthread_t tid;

    pthread_create(&tid, NULL, print_message, NULL);

    pthread_join(tid, NULL);

    return 0;
}
```

Explanation

1. The program starts execution in the main thread.
2. `pthread_create()` creates a new thread.
3. The newly created thread executes the function `print_message()`.
4. The message "Hello from Thread" is displayed.

5. **pthread_join()** causes the main thread to wait until the created thread finishes execution.
6. **After the thread terminates, the program ends.**

Advantages of Pthreads

1. **Supports concurrent execution.**
2. **Improves CPU utilization.**
3. **Allows resource sharing among threads.**
4. **Faster than creating separate processes.**
5. **Suitable for multiprocessor systems.**

7. Describe the Differences among Short-Term, Medium-Term and Long-Term Scheduling.

Schedulers

A process migrates among various scheduling queues throughout its lifetime. For scheduling purposes, the operating system selects processes from these queues using Schedulers.

There are three types of schedulers:

1. **Long-Term Scheduler**
2. **Short-Term Scheduler**
3. **Medium-Term Scheduler**

Differences among Long-Term, Medium-Term and Short-Term Scheduling

Feature	Long-Term Scheduler	Medium-Term Scheduler	Short-Term Scheduler
Also Called	Job Scheduler	Swapper	CPU Scheduler
Purpose	Selects processes from secondary storage and loads them into main memory	Removes processes from memory and reintroduces them later	Selects a process from the ready queue and allocates CPU
State Transition	New → Ready	Swapping Operations	Ready → Running
Main Function	Controls admission of processes into memory	Performs swapping and memory management	Allocates CPU for execution
Execution Frequency	Executes less frequently	Executes occasionally	Executes very frequently
Speed	Relatively slow	Moderate	Must be very fast
Controls	Degree of multiprogramming	Memory utilization	CPU utilization
Location of Processes	Secondary Storage → Main Memory	Main Memory ↔ Secondary Storage	Ready Queue → CPU
System Impact	Determines number of processes in memory	Reduces memory load and improves performance	Improves CPU efficiency
Example Activity	Admitting a new job into memory	Swapping out a suspended process	Choosing next process for CPU execution

Additional Points

- Long-Term Scheduler maintains a proper mix of CPU-bound and I/O-bound processes.

- Medium-Term Scheduler performs swapping, temporarily removing processes from memory and bringing them back later.
- Short-Term Scheduler is invoked most frequently and therefore must be very fast.

8. What is Scheduling Queue? Explain Different Types of Scheduling Queues with the Help of a Diagram.

Scheduling Queues

As processes enter the system, they are placed in various queues maintained by the operating system. These queues help the scheduler select processes for execution.

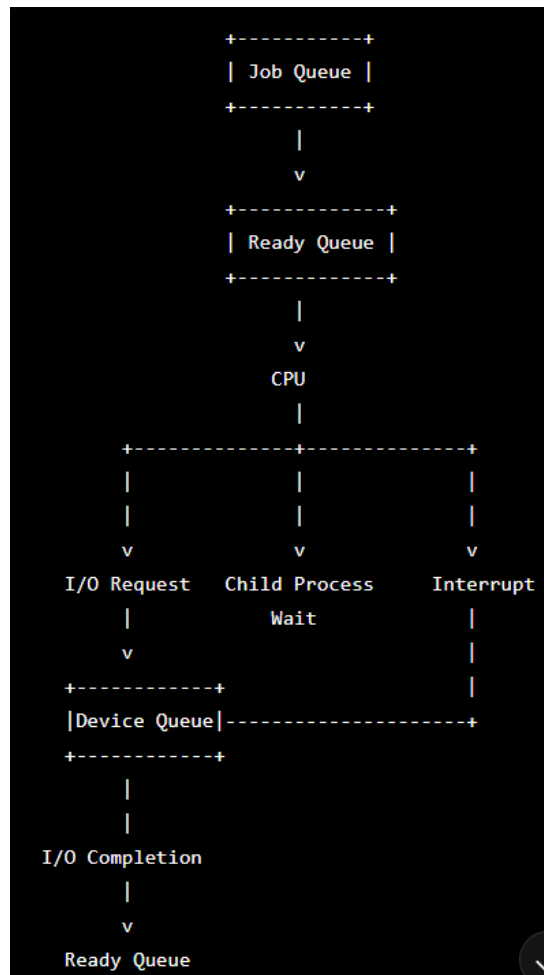
There are three types of scheduling queues:

1. Job Queue
2. Ready Queue
3. Device Queue

Types of Scheduling Queues

Queue	Description
Job Queue	Consists of all processes in the system. As processes enter the system, they are placed in the job queue.
Ready Queue	Contains processes residing in main memory that are ready and waiting for CPU allocation.
Device Queue	Each I/O device has its own device queue containing processes waiting for that particular device.

Scheduling Queue Diagram



Process Flow in Scheduling Queues

- A new process enters the Job Queue.
- It is admitted into the Ready Queue.
- The CPU Scheduler selects a process from the Ready Queue.
- If the process requests I/O, it moves to the Device Queue.
- After I/O completion, it returns to the Ready Queue.
- If interrupted, it is moved back to the Ready Queue.
- The cycle continues until the process terminates.

9. What is the Criteria for Comparing CPU Scheduling Algorithms?

Different CPU scheduling algorithms have different properties. The choice of a scheduling algorithm may favor one class of processes over another. Therefore, certain criteria are used to compare CPU scheduling algorithms.

Scheduling Criteria

Criterion	Description
CPU Utilization	CPU utilization refers to keeping the CPU as busy as possible. CPU utilization can range from 0 to 100%. In a real system, it should range from 40% to 90%.
Throughput	Throughput is the number of processes completed per unit time. Higher throughput indicates better performance.
Turnaround Time	Turnaround time is the interval from the submission of a process to its completion. It includes waiting time, execution time, and I/O time.
Waiting Time	Waiting time is the total amount of time a process spends waiting in the ready queue.
Response Time	Response time is the time from submission of a request until the first response is produced. It is mainly used in interactive systems.

Desired Goals

- CPU Utilization → Maximize
- Throughput → Maximize
- Turnaround Time → Minimize
- Waiting Time → Minimize
- Response Time → Minimize

10. Compare Priority Scheduling with Shortest Job First (SJF) Scheduling.

Feature	Shortest Job First (SJF) Scheduling	Priority Scheduling
Basis of Selection	Process with the shortest CPU burst time is selected first.	Process with the highest priority is selected first.
Scheduling Factor	CPU burst time.	Priority value assigned to the process.
Objective	Minimizes average waiting time.	Executes more important processes first.
Optimality	SJF is optimal because it gives the minimum average waiting time for a given set of processes.	Not necessarily optimal in terms of waiting time.
Preemption	Can be preemptive or non-preemptive.	Can be preemptive or non-preemptive.
Starvation	Long processes may suffer starvation if shorter jobs continue to arrive.	Low-priority processes may suffer starvation if high-priority processes keep arriving.
Difficulty	Difficult to know the exact length of the next CPU burst.	Priority values must be assigned accurately.
Selection Rule	Smaller burst time gets higher preference.	Higher priority gets higher preference.

SJF Scheduling

- The CPU is assigned to the process having the smallest next CPU burst.
- If two processes have the same burst time, FCFS scheduling is used.
- SJF provides the minimum average waiting time.

Priority Scheduling

- Each process is assigned a priority.
- The CPU is allocated to the process with the highest priority.

- Equal-priority processes are scheduled using FCFS.
- Starvation may occur for low-priority processes.

14. Write Short Notes on Multilevel Queue Scheduling and Multilevel Feedback Queue Scheduling.

1. Multilevel Queue Scheduling (MLQ)

Multilevel Queue Scheduling divides the ready queue into several separate queues.

Each queue is assigned to a particular class of processes such as:

- System Processes
- Interactive Processes
- Interactive Editing Processes
- Batch Processes
- Student Processes

Each queue has its own scheduling algorithm. For example:

- Foreground Queue → Round Robin
- Background Queue → FCFS

Features

- Processes are permanently assigned to a queue when they enter the system.
- Processes are not allowed to move from one queue to another.
- Each queue has absolute priority over lower-priority queues.
- Lower-priority processes execute only when higher-priority queues are empty.

Advantages

- Simple to implement.
- Different scheduling algorithms can be used for different classes of processes.

Disadvantages

- Inflexible scheduling.
- Starvation of lower-priority queues may occur.
- Processes cannot move between queues.

2. Multilevel Feedback Queue Scheduling (MLFQ)

Multilevel Feedback Queue Scheduling overcomes the limitations of Multilevel Queue Scheduling.

In this algorithm, processes are allowed to move between queues.

Features

- Processes are separated according to their CPU burst characteristics.
- A process that uses too much CPU time is moved to a lower-priority queue.
- A process waiting too long in a lower-priority queue can be moved to a higher-priority queue.
- Aging is used to prevent starvation.

Example

Consider three queues:

- Queue0
- Queue1
- Queue2

Operation:

- New processes enter Queue0.
- Queue0 uses a time quantum of 8 ms.
- If a process does not finish, it moves to Queue1.
- Queue1 uses a time quantum of 16 ms.
- If still unfinished, it moves to Queue2.
- Queue2 follows FCFS scheduling.
- Queue0 has higher priority than Queue1, and Queue1 has higher priority than Queue2.

Advantages

- Flexible scheduling.
- Supports process movement between queues.
- Prevents starvation using aging.
- Provides better response for short processes.

Disadvantages

- More complex to implement.
- Scheduling overhead is higher.

Difference Between MLQ and MLFQ

Feature	Multilevel Queue (MLQ)	Multilevel Feedback Queue (MLFQ)
Process Movement	Not allowed	Allowed
Flexibility	Inflexible	Flexible
Starvation	Possible	Reduced by aging
Complexity	Simple	More Complex
Priority Change	Fixed	Dynamic
Performance	Lower	Better



SAQ's

1. Define Process and Thread.

Process

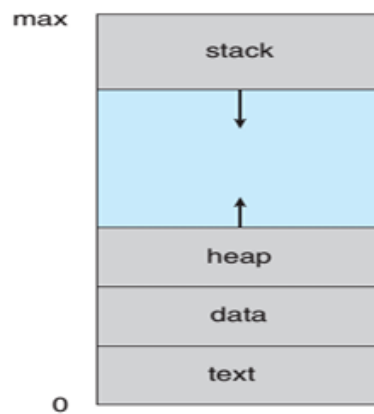
A process is a program in execution. It is the basic unit of work in a modern operating system and requires resources such as CPU time, memory, files, and I/O devices for execution.

Thread

A thread is the basic unit of CPU utilization. It consists of a thread ID, program counter, register set, and stack. Multiple threads can exist within a process and share the same resources.

2. Draw the Diagram of Structure of Process in Memory and Explain.

Process Structure in Memory



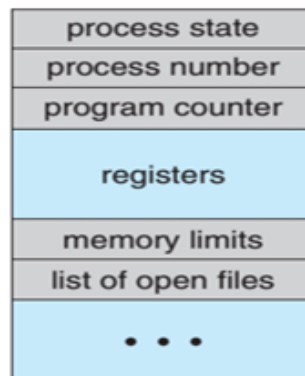
- Text Section – Contains program code.
- Data Section – Contains global variables.
- Heap – Stores dynamically allocated memory.
- Stack – Stores local variables, function parameters, and return addresses.

3. What is Process Control Block? Draw its Structure.

Process Control Block (PCB)

PCB is a data structure maintained by the operating system to store information about a process. It acts as a repository of all information related to a process.

Structure: draw as it is by mentioning the all fields of pcb in 4th question



4. What is PCB? List its Fields.

Fields of PCB

1. Process State
2. Program Counter
3. CPU Registers
4. CPU Scheduling Information
5. Memory Management Information
6. Accounting Information
7. I/O Status Information

5. What is Context Switching?

Context Switching is the process of switching the CPU from one process to another.

During context switching:

- The current process state is saved in its PCB.
- The state of the next process is restored from its PCB.

Context-switch time is pure overhead because no useful work is done during switching.

6. Give the Actions Taken by a Kernel During Context Switch.

1. Save the context of the currently running process into its PCB.
2. Save CPU registers, process state, and memory-management information.
3. Load the saved context of the next scheduled process.
4. Restore CPU registers and process state.
5. Transfer CPU control to the new process.

7. Draw the Process State Transition Diagram.

→Waiting → Ready

New → Ready → Running →Ready

→Terminated

States:

- New
- Ready
- Running
- Waiting
- Terminated

8. List the Operations on Processes.

The two major operations on processes are:

1. Process Creation
2. Process Termination

9. Explain the Role of Init Process in UNIX and Linux Systems.

- The init process always has PID = 1.
- It acts as the root parent process for all user processes.
- If a parent terminates before its child, the child becomes an orphan process.
- The init process adopts orphan processes and periodically invokes wait() to collect their exit status.

10. What is a Zombie Process and Orphan Process?

Zombie Process

A process that has terminated but whose parent has not yet called wait() is called a Zombie Process.

Orphan Process

A process whose parent terminates before it does is called an Orphan Process. The init process becomes its new parent.

11. Differentiate Between Process and Thread.

Process	Thread
Program in execution	Smallest unit of CPU execution
Has its own memory space	Shares memory of the process
Heavyweight	Lightweight
Creation is costly	Creation is less costly
Communication is slower	Communication is faster

12. What are User Threads and Kernel Threads?

User Threads

- Managed by a thread library.
- Kernel is unaware of them.
- Faster to create and manage.

Kernel Threads

- Managed directly by the operating system.
- Scheduled by the kernel.
- Support true parallel execution.

13. Distinguish Preemptive and Non-Preemptive Scheduling.

Preemptive Scheduling	Non-Preemptive Scheduling
CPU can be taken away from a process	CPU cannot be taken away until completion
Better response time	Poor response time
Suitable for time-sharing systems	Suitable for batch systems
More overhead	Less overhead

14. List Three Main Thread Libraries in Use Today.

1. **POSIX Pthreads**
2. **Windows Threads**
3. **Java Threads**

15. Approaches to Multiple Processor Scheduling (SMP and ASMP)

Asymmetric Multiprocessing (ASMP)

- Master-worker relationship exists.
- One processor handles scheduling and system activities.
- Other processors execute user code.

Symmetric Multiprocessing (SMP)

- Each processor performs self-scheduling.
- All processors are treated equally.
- Better load sharing and performance.

16. Define Soft Affinity and Hard Affinity.

Soft Affinity

The operating system attempts to keep a process on the same processor, but migration is possible.

Hard Affinity

A process is restricted to run only on a specified set of processors.

17. What is Load Balancing? Explain Push Migration and Pull Migration.

Load Balancing

Load balancing distributes workload evenly among processors in an SMP system.

Push Migration

A task is moved from an overloaded processor to an idle processor.

Pull Migration

An idle processor pulls a waiting task from a busy processor.

18. What is Memory Stall?

Memory stall is the delay experienced by a processor when it must wait for data to be fetched from memory before continuing execution.

1. CPU-I/O Burst Cycle

CPU-I/O Burst Cycle

Process execution consists of a cycle of CPU execution and I/O wait.

- A process begins execution with a CPU Burst.
- This is followed by an I/O Burst.
- Processes alternate between CPU bursts and I/O bursts throughout their execution.
- The final CPU burst ends with a request to terminate execution.

Diagram

CPU Burst → I/O Burst → CPU Burst → I/O Burst → CPU Burst → Termination

Explanation

- CPU Burst: Time during which the process executes instructions on the CPU.
- I/O Burst: Time during which the process waits for an I/O operation to complete.
- The first and last bursts of a process are always CPU bursts.

2. CPU Scheduler

CPU Scheduler

Whenever the CPU becomes idle, the operating system must select one of the processes in the ready queue for execution.

The component responsible for this selection is called the CPU Scheduler or Short-Term Scheduler.

Functions of CPU Scheduler

1. Selects a process from the ready queue.
2. Allocates the CPU to the selected process.

3. Improves CPU utilization.
4. Supports multiprogramming and multitasking.

Importance

- Ensures efficient utilization of CPU resources.
- Reduces waiting time and turnaround time.
- Helps achieve better system performance.

3. Dispatcher and Dispatch Latency

Dispatcher

The dispatcher is the module that gives control of the CPU to the process selected by the short-term scheduler.

Functions of Dispatcher

1. Switching context.
2. Switching to user mode.
3. Jumping to the correct location in the user program to restart execution.

Dispatch Latency

Dispatch Latency is the time taken by the dispatcher to stop one process and start another process.

Characteristics

- Should be as small as possible.
- Large dispatch latency reduces system performance.

4. Processor Affinity

Processor Affinity

Processor Affinity is the tendency of a process to run on the same CPU on which it was previously executed.

The processor cache contains data recently used by the process. If the process moves from one processor to another, the cache contents must be reloaded, which decreases performance. Therefore, most SMP systems try to keep a process running on the same processor.

Need for Processor Affinity

- Reduces cache misses.
- Improves CPU performance.
- Reduces cache invalidation overhead.
- Increases system efficiency.

Types of Processor Affinity

Soft Affinity	Hard Affinity
Operating system attempts to keep a process on the same processor.	Process is restricted to run on a specific processor or set of processors.
Process migration is possible.	Process migration is restricted.
Flexible approach.	Strict approach.

5. User Threads vs Kernel Threads

Difference Between User Threads and Kernel Threads

User Threads	Kernel Threads
Managed by thread library	Managed by operating system
Kernel is unaware of threads	Kernel is aware of threads
Faster creation and switching	Slower creation and switching
Lower overhead	Higher overhead
Blocking system call may block entire process	Other threads can continue execution
Implemented in user space	Implemented in kernel space

1. Burst Time (BT)

Burst Time is the amount of CPU time required by a process to complete its execution.

2. Arrival Time (AT)

Arrival Time is When does the process enter the Ready Queue?

3. Completion Time (CT)

Completion Time means At what time did the process finish?

4. Turnaround Time (TAT)(Waiting+Running)

Turnaround Time means How much total time the process spent in the system.

Formula: $TAT = Completion\ Time - Arrival\ Time$

5. Waiting Time (WT)

This is How long the process waited for CPU.

Formula: $WT = Turnaround\ Time - Burst\ Time$

Turnaround Time:

$$TAT = CT - AT$$

Waiting Time:

$$WT = TAT - BT$$

Shortcut for FCFS

If all Arrival Times are 0, then:

- $CT =$ Keep adding Burst Times one after another.
- $TAT = CT$
- $WT = TAT - BT$

Golden Rule of SJF:

Choose the shortest job only among the processes that have already arrived.

Shortcut:

Algorithm	Compare What?
FCFS	Arrival Order
SJF	Burst Time
Priority	Priority Number

★ The Golden Rules of Round Robin

1. Execute the first process in the queue.
2. Give it Time Quantum (TQ).
3. If it finishes, remove it.
4. If it doesn't finish, subtract the TQ from its BT and send it to the back of the queue.

5. **Repeat until all processes finish.**